FSBM
FACULTE DES SCIENCES BEN M'SIK
UNIVERSITÉ HASSAN II DE CASABLANCA

Infrastructure DevOps

Pour Smart Booking

Présenté par:

Chouiekh majda
El Filali Malak
Khamalli Nour
El Kassmi Nadia

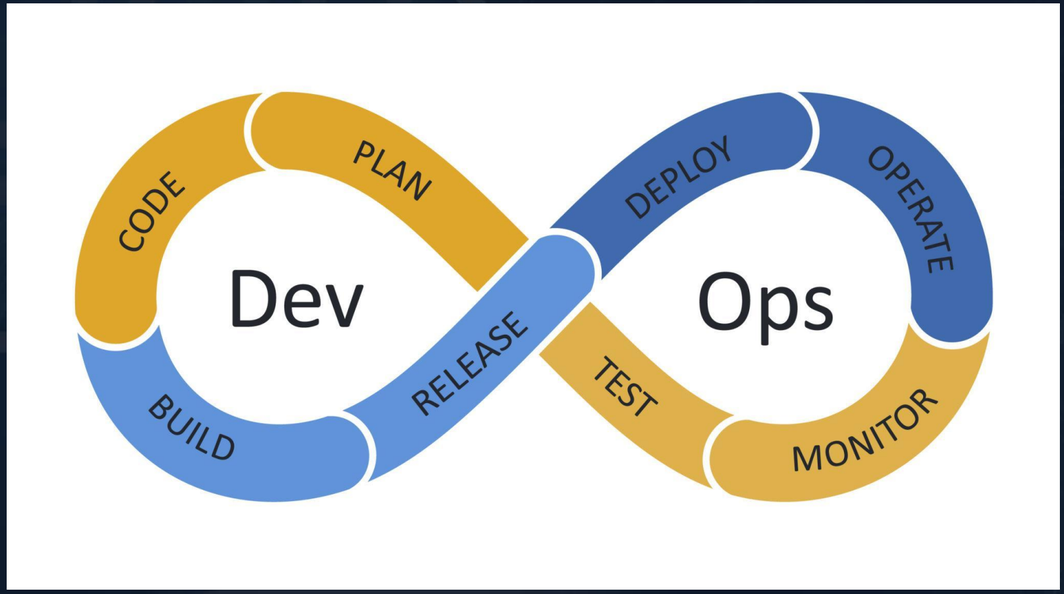
Encadré par:

Pr.Benabbou Faouzia

Date de présentation : **24 / 03 / 2026**

Introduction & Contexte

- Les applications modernes nécessitent des déploiements rapides
- Le déploiement manuel est lent et sujet aux erreurs
- Le Cloud permet la scalabilité et la disponibilité
- Le DevOps automatise le cycle de vie des applications
- Utilisation d’outils : Docker, Kubernetes, CI/CD



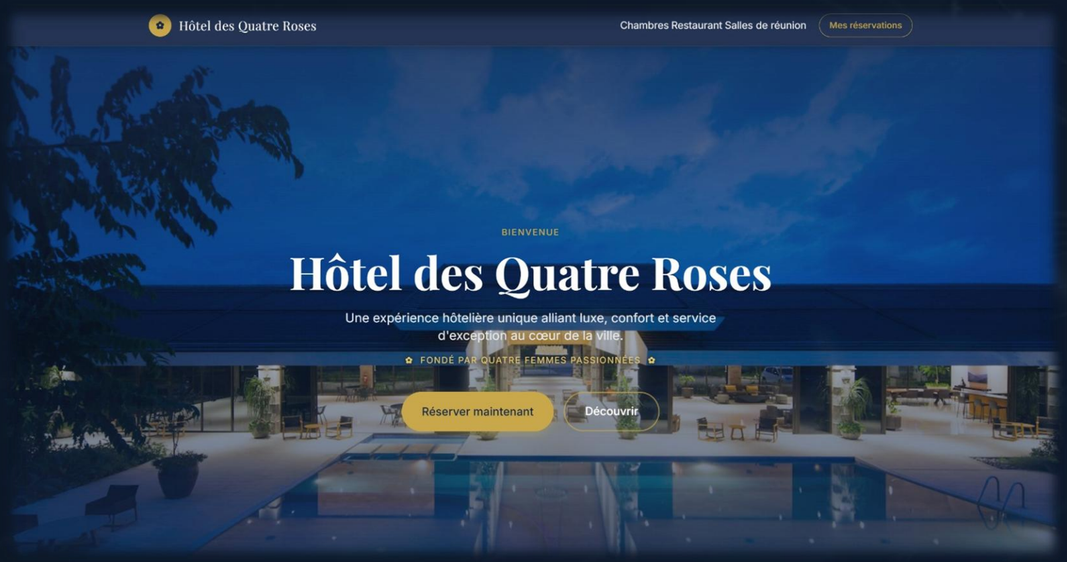
Objectifs du projet

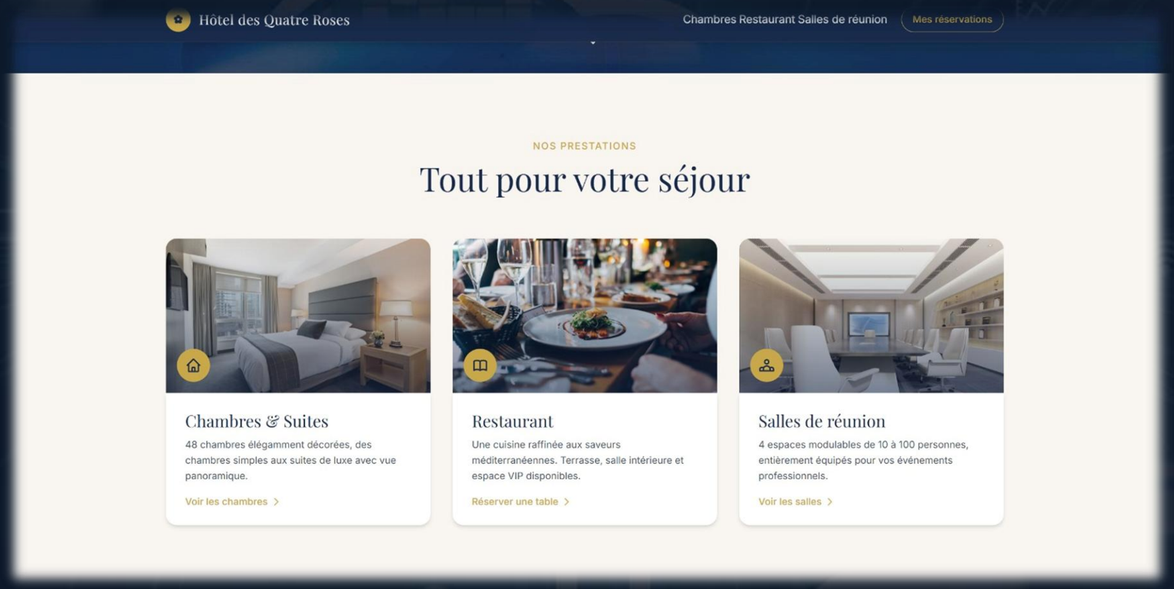
- Automatiser entièrement le déploiement d'une application sur le cloud
- Mettre en place un cluster Kubernetes
- Conteneuriser l'application avec Docker
- Implémenter un pipeline CI/CD
- Déployer une application réelle sur le cloud

SmartBooking – Application de réservation d'hôtel

Flask *(Python 3.1)*

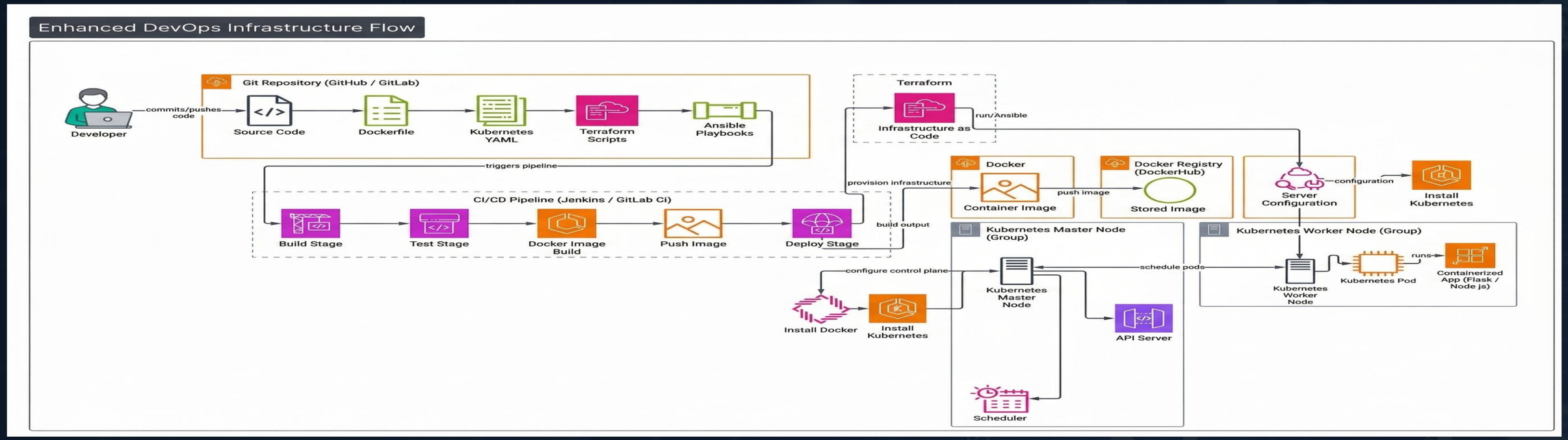
Sqlite *(Base de données)*





- L'application permet aux utilisateurs de consulter les chambres, le restaurant et les salles de réunion.
- L'utilisateur recherche une disponibilité, remplir un formulaire et effectuer une réservation en ligne de manière simple et rapide.
- Interface utilisateur intuitive et responsive.

Architecture globale du système



Gestion du code

- Le développeur pousse le code sur GitHub
- Le projet contient Dockerfile et fichiers Kubernetes

Pipeline CI/CD

- Le pipeline est déclenché automatiquement
- Il exécute build, test et déploiement

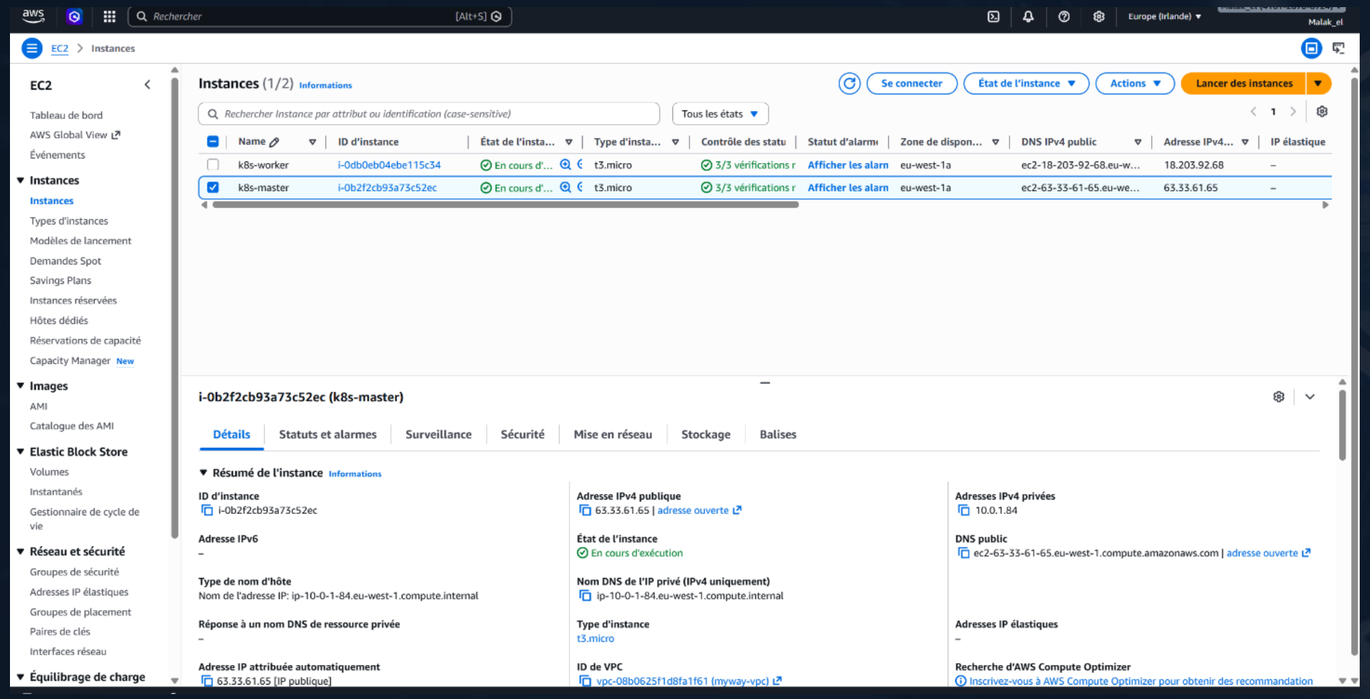
Infrastructure

- Terraform crée les ressources sur AWS
- Ansible configure les serveurs automatiquement

Déploiement

- L'application est conteneurisée avec Docker
- Kubernetes déploie les pods sur le cluster

Infrastructure AWS



- Utilisation d’AWS comme plateforme cloud
- Mise en place d’un environnement sécurisé (VPC, Security Groups)
- Création de deux instances EC2 :
- Master Node
- Worker Node
- Infrastructure destinée à héberger le cluster Kubernetes
- Provisionnement automatisé avec Terraform

Terraform

- Automatisation de la création de l'infrastructure
- Définition des ressources via des fichiers .tf
- Création des instances EC2 (Master / Worker)
- Configuration du réseau et des règles de sécurité
- Reproductibilité et déploiement rapide

```
malak@Malak MINGW64 ~/Desktop/k8s-cluster-starter/terraform
$ terraform apply
data.aws_ami.amazon_linux_2: Reading...
data.aws_ami.amazon_linux_2: Read complete after 1s [id=ami-08ae1035a3a101a88]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbo
ls:
  + create

Terraform will perform the following actions:

  # aws_instance.k8s_master will be created
  + resource "aws_instance" "k8s_master" {
      + ami                                  = "ami-08ae1035a3a101a88"
      + arn                                  = (known after apply)
      + associate_public_ip_address          = (known after apply)
      + availability_zone                    = (known after apply)
      + cpu_core_count                       = (known after apply)
      + cpu_threads_per_core                 = (known after apply)
```

```
malak@Malak MINGW64 ~/Desktop/k8s-cluster-starter/terraform
$ terraform output
backend_url = "http://63.33.61.65:30500"
frontend_url = "http://63.33.61.65:30080"
grafana_url = "http://63.33.61.65:30030"
health_check_url = "http://63.33.61.65:30500/api/health"
master_instance_id = "i-0b2f2cb93a73c52ec"
master_private_ip = "10.0.1.84"
master_public_ip = "63.33.61.65"
prometheus_url = "http://63.33.61.65:30090"
ssh_command_master = "ssh -i ../ssh/k8s-key ec2-user@63.33.61.65"
ssh_command_worker = "ssh -i ../ssh/k8s-key ec2-user@18.203.92.68"
worker_instance_id = "i-0db0eb04ebe115c34"
worker_private_ip = "10.0.1.56"
worker_public_ip = "18.203.92.68"
```

Ansible – Configuration des serveurs

4 Playbooks

- Automatisation de la configuration des machines
- Installation de Docker et Kubernetes
- Initialisation du nœud Master
- Ajout du nœud Worker au cluster
- Exécution via des playbooks Ansible

```
# Install kubeadm, kubelet, kubectl on both nodes
ansible-playbook playbooks/02-install-kubernetes.yml
```

```
malak@Malak:/mnt/c/Users/malak/Desktop/k8s-cluster-starter/ansible$ ansible-playbook playbooks/01-install-docker.yml

PLAY [Installer Docker sur tous les noeuds Kubernetes] ***********************************************************************************
****************************************

TASK [Gathering Facts] ********************************************************************************************************************
****************************************
[WARNING]: Platform linux on host k8s-master is using the discovered Python interpreter at /usr/bin/python3.7, but future installation
of another Python interpreter could
change the meaning of that path. See https://docs.ansible.com/ansible-core/2.16/reference_appendices/interpreter_discovery.html for mor
e information.
ok: [k8s-master]
[WARNING]: Platform linux on host k8s-worker is using the discovered Python interpreter at /usr/bin/python3.7, but future installation
of another Python interpreter could
change the meaning of that path. See https://docs.ansible.com/ansible-core/2.16/reference_appendices/interpreter_discovery.html for mor
e information.
ok: [k8s-worker]
```

Exécution des playbooks Ansible pour l'installation de Docker et Kubernetes

```
# Initialize the master (kubeadm init + Flannel CNI)
ansible-playbook playbooks/03-init-master.yml
```

```
malak@Malak:/mnt/c/Users/malak/Desktop/k8s-cluster-starter/ansible$ ansible-playbook playbooks/04-join-worker.yml

PLAY [Joindre le Worker au Cluster Kubernetes] ********************************************************************************************
****************************************

TASK [Gathering Facts] ********************************************************************************************************************
****************************************
[WARNING]: Platform linux on host k8s-worker is using the discovered Python interpreter at /usr/bin/python3.7, but future installation
of another Python interpreter could
change the meaning of that path. See https://docs.ansible.com/ansible-core/2.16/reference_appendices/interpreter_discovery.html for mor
e information.
ok: [k8s-worker]
```

Initialisation du nœud master et ajout du nœud worker au cluster

```
[master]
k8s-master ansible_host=63.33.61.65 ansible_user=ec2-user

[workers]
k8s-worker ansible_host=18.203.92.68 ansible_user=ec2-user
```

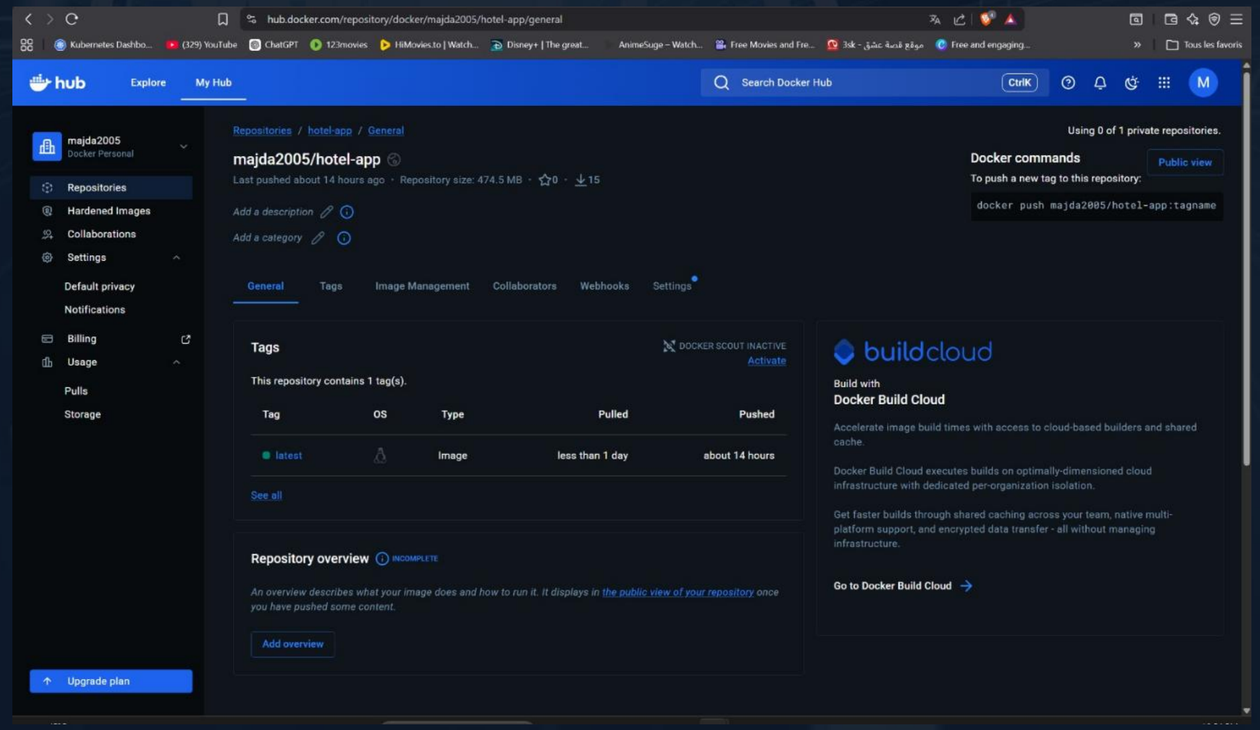
Fichier inventory Ansible définissant les nœuds du cluster

Docker - Conteneurisation

- Conteneurisation de l’application
- Création d’une image Docker via Dockerfile
- Utilisation d’un Dockerfile
- Intégration des dépendances de l’application
- Portabilité entre différents environnements
- Préparation pour le déploiement Kubernetes

```
PS C:\Users\malak\Desktop\k8s-cluster-starter> docker build -t hotel-app .
```

Création de l’image Docker

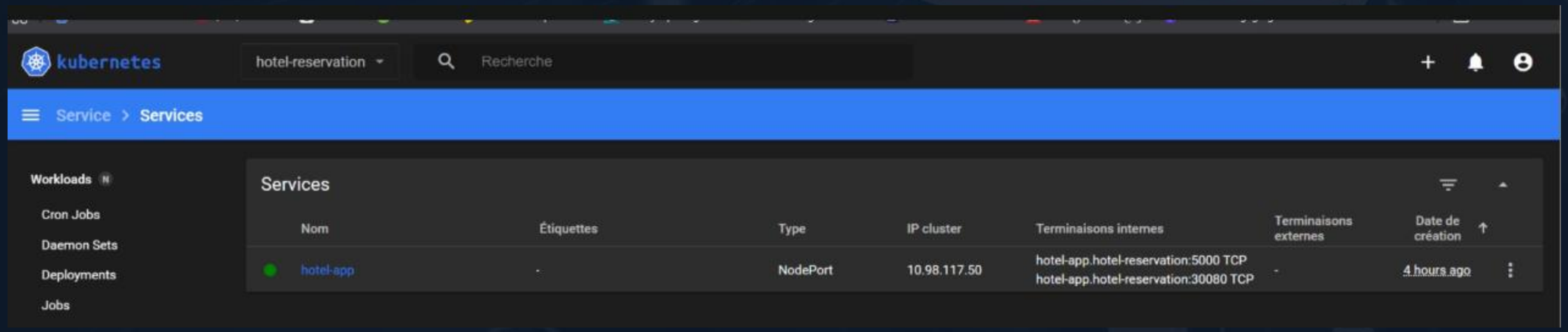


Kubernetes – Orchestration

- Déploiement des conteneurs sur le cluster
- Gestion des pods et des services
- Cluster composé d'un Master et d'un Worker
- Orchestration automatique des applications
- Supervision via Kubernetes Dashboard

```
malak@Malak:/mnt/c/Users/malak/Desktop/k8s-cluster-starter$ kubectl --kubeconfig ~/.kube/k8s-config get nodes
NAME                                        STATUS   ROLES           AGE   VERSION
ip-10-0-1-56.eu-west-1.compute.internal     Ready    <none>          16m   v1.29.0
ip-10-0-1-84.eu-west-1.compute.internal     Ready    control-plane   17m   v1.29.0
```

kubectl get nodes



Kubernetes Dashboard

Manifestes Kubernetes

Déploiement via fichiers YAML

deployment.yml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: hotel-app
  namespace: hotel-reservation
spec:
  replicas: 1
  selector:
    matchLabels: {app: hotel-app}
  template:
    spec:
      containers:
      - name: hotel-app
        image: majda2005/hotel-app:latest
        ports:
        - containerPort: 5000
        env:
        - name: SECRET_KEY
          value: hotel-secret-key-prod-2024
        volumeMounts:
        - mountPath: /app/instance
          name: hotel-db
```

service.yml

```yaml
apiVersion: v1
kind: Service
metadata:
  name: hotel-service
  namespace: hotel-reservation
spec:
  type: NodePort
  ports:
  - port: 5000
    nodePort: 30080
  selector:
    app: hotel-app
```

storage.yml (PV + PVC)

```yaml
# PersistentVolume
kind: PersistentVolume
spec:
  capacity:
    storage: 1Gi
  hostPath:
    path: /data/hotel-db
---
# PersistentVolumeClaim
kind: PersistentVolumeClaim
spec:
  accessModes:
  - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
```

Manifestes Kubernetes

- Fichiers YAML pour définir les ressources
- deployment.yml : déploiement de l'application
- service.yml : exposition via **NodePort (30080)**
- storage.yml : gestion du stockage

Pipeline CI/CD – GitHub Actions

Déclencheur : git push → branche main | Fichier : .github/workflows/cicd.yml

Job 1 : Test

- pip install -r requirements.txt
- Vérifie: from app import create_app
- pytest (test_routes.py, test_availability.py)
- Validation syntaxique Python

Job 2 : Build & Push

- docker buildx build --no-cache
- Tag: majda2005/hotel-app:latest
- Tag: majda2005/hotel-app:<sha7>
- Push DockerHub (DOCKERHUB_TOKEN)

Job 3 : Deploy

- SCP: copie k8s/ → master SSH
- kubectl apply -f namespace/pv/pvc/deploy/svc
- kubectl set image (SHA exact)
- rollout status (timeout 5min)
- kubectl exec → python seed.py

GitHub Secrets requis

DOCKERHUB_USERNAME	DOCKERHUB_TOKEN	SSH_PRIVATE_KEY	MASTER_IP
majda2005	*Token DockerHub*	*Clé privée EC2 (myway-key)*	*63.33.61.65*

Démonstration

Application Flask

http://63.33.61.65:30080/

Navigation: Chambres · Restaurant · Salles
Réservation live + vérification disponibilité

Kubernetes Cluster

kubectl get pods -n hotel-reservation

Pods Running · Rolling update · Logs temps réel

Pipeline CI/CD

GitHub Actions → Push commit

Test → Build → Push DockerHub → Deploy SSH automatique

Kubernetes Dashboard

kubectl proxy → localhost:8001

Supervision pods · Logs · Namespaces · Services

Conclusion & Perspectives

Une infrastructure DevOps complète, automatisée de bout en bout,
de l'Infrastructure as Code jusqu'au déploiement en production sur Kubernetes.

Compétences acquises

- ✓ Infrastructure as Code (Terraform + Ansible) — reproductible, idempotent
- ✓ Conteneurisation Docker — portabilité totale de l'application
- ✓ Orchestration Kubernetes — haute disponibilité et rolling updates
- ✓ CI/CD automatisé — déploiement en < 5 min
- ✓ Kubernetes Dashboard — supervision visuelle des pods et services

Chiffres clés

2	Nœuds EC2 AWS
4	Playbooks Ansible
5	Manifestes Kubernetes
3	Etapes CI/CD
<5min	Déploiement
100%	Automatisation

Merci pour votre attention